AI-Powered Universal Desktop & Web Software Testing Agent

Product Requirements, UX, Architecture, AI Perception, Execution, Testing and Roadmap

Working project specification • Version 1.0

1. Executive Summary

The proposed product is an AI-powered automated software testing agent that can operate applications from what is visually available on the screen. Unlike traditional browser-only automation that depends primarily on DOM selectors, the system is designed to work with both web and desktop applications. It perceives screenshots, identifies UI elements and their relationships, reasons about natural-language test instructions, performs mouse and keyboard actions, verifies outcomes, handles ambiguity and changes, and produces evidence-rich test reports.

The central product idea is: a user describes what should be tested; the system converts that intent into an executable test plan; observes the application like a human; interacts with it through mouse and keyboard control; verifies the expected result; and explains the outcome.

```
Describe → Plan → Perceive → Identify → Act → Verify → Explain → Report
```

2. Product Vision

The long-term vision is a universal visual software-testing agent that can test applications without requiring the customer to write brittle selectors or automation code. The system should operate through a human-like interaction layer while using AI internally for perception, semantic understanding, planning, recovery, and analysis.

The product should not be positioned as simply "an AI that controls the mouse." The customer-facing value proposition is:

```
"Describe what you want to test. The AI observes the application like a human, interacts with it,
verifies the result, and gives you evidence when something goes wrong."
```

3. Problem Statement

- Traditional UI automation often depends on selectors, DOM structure, accessibility trees, IDs, or application-specific automation APIs.
- Desktop applications may not expose a DOM at all.
- UI redesigns can break selector-based tests.
- Identical text can refer to different controls or non-interactive content.
- Icon-only controls are difficult for OCR-based systems.
- Coordinate-only automation is brittle when windows, layouts, resolution, or DPI change.
- A useful AI testing system must understand visual context, semantic roles, relationships, and user intent rather than matching text alone.
- Test failures need actionable explanations and visual evidence rather than only a generic PASS/FAIL result.

4. Target Users

User	Primary Need	Expected Experience
QA Engineer	Create, run, debug and maintain tests	Detailed control, evidence, logs and recovery
Developer	Run regression checks quickly	Simple natural-language tests and CI results
Product/Business Tester	Test without automation	Describe expected behavior in

	knowledge	plain language
QA Lead	Understand quality trends	Suites, reports, failure analytics and history
Enterprise Team	Scale testing across environments	Parallel workers, permissions, integrations and security

5. Product Scope

5.1 In Scope for the Core Product

- Natural-language test creation.
- AI-generated executable test plans.
- Test review and editing before execution.
- Screenshot-based visual perception.
- OCR for text detection.
- Computer-vision analysis for regions, shapes and controls.
- VLM-based semantic interpretation and ambiguity resolution.
- UI element graph / structured element map.
- Mouse and keyboard execution.
- Smart waiting and state detection.
- Visual and semantic assertions.
- Failure detection and classification.
- Screenshots and execution evidence.
- Detailed PASS/FAIL/HEALED results.
- Test suites and regression execution.
- Optional recording of human interactions.
- Self-healing / recovery from reasonable UI changes.

5.2 Later Scope

- Isolated desktop sessions.
- Virtual machines and remote execution workers.
- Parallel test execution.
- CI/CD integrations.
- Scheduling.
- Secrets management and enterprise controls.
- Team collaboration and permissions.
- Jira/GitHub/Slack/Teams integrations.
- Cross-environment execution.
- Cloud worker pools and centralized dashboards.

6. Functional Requirements

6.1 Project and Application Management

- The system shall allow a user to create and manage testing projects.
- The system shall allow the user to define or launch an application under test.
- The system shall preserve test cases, suites, execution history and evidence within the project.

- The system shall support configurable execution environments.

6.2 Natural-Language Test Creation

- The system shall accept natural-language test descriptions.
- The system shall convert a natural-language description into ordered executable steps.
- The system shall infer actions, targets, expected states and assertions.
- The user shall be able to review and edit the generated test before execution.

6.3 Test Recording

- The system should optionally record user mouse and keyboard interactions.
- The system should convert recorded interactions into reusable semantic test steps.
- The user should be able to edit generated steps and add assertions.

6.4 Perception

- The system shall capture screenshots from the test environment.
- The system shall detect visible text and bounding boxes.
- The system shall detect or infer UI regions and controls.
- The system shall identify icons and icon-like controls.
- The system shall infer visual and semantic properties of UI elements.
- The system shall maintain stable element IDs within an execution state.
- The system shall represent spatial and semantic relationships between elements.

6.5 Test Action Execution

- The system shall support click, double-click, right-click, mouse movement, drag, scroll, typing, keyboard shortcuts, keypresses, copy/paste and text selection.
- Actions shall preferably target semantic elements rather than hard-coded coordinates.
- The executor shall calculate a safe point inside the selected element's bounding box.
- The executor shall verify that the application is in the expected state before or after critical actions.

6.6 Assertions

- The system shall support visibility, presence, absence, text, value, state, visual and workflow assertions.
- The system shall verify expected results after actions.
- The system shall capture evidence when an assertion fails.
- The system should support AI-assisted visual assertions for complex screen states.

6.7 Failure Handling

- The system shall detect missing elements, failed actions, failed assertions, application crashes, timeouts, vision ambiguity and unknown failures.
- The system shall stop, retry, recover or request intervention according to configured policy.
- The system shall explain likely causes of failures.
- The system shall preserve screenshots and execution logs for investigation.

6.8 Self-Healing

- The system should identify likely replacement elements when an expected element changes.
- The system shall record when a test step was healed.
- The system should expose the original target, replacement target, reasoning and confidence.
- The system should never silently hide a healing event.

6.9 Test Management and Reporting

- The system shall support individual test cases and test suites.
- The system shall support running individual tests, selected tests, suites and regression collections.
- The system shall provide execution history.
- The system shall provide step-level PASS/FAIL/HEALED results.
- The system shall provide screenshots and optionally video evidence.
- The system shall generate human-readable reports.

7. Non-Functional Requirements

7.1 Accuracy

The system should reliably identify the intended UI element even when multiple elements have identical or similar text. Accuracy targets should be benchmarked on a representative dataset before being made contractual.

7.2 Coordinate Precision

The system shall click within a safe interactive area of the selected target. The product should prefer a safe interior click point rather than assuming the exact mathematical center is always correct.

7.3 Reliability and Low Flakiness

Repeated executions against the same application state should produce consistent outcomes. The system should avoid unnecessary timing dependencies and use state-based waiting.

7.4 Robustness

- Tolerate reasonable window movement and resizing.
- Tolerate reasonable screen-resolution and DPI changes.
- Tolerate small visual/layout changes.
- Handle loading states, dialogs and transient UI.
- Recover from temporary model or perception failures where possible.

7.5 Performance

Perception, reasoning and execution should be fast enough for interactive testing. Exact latency targets should be established by benchmarking on the selected hardware and model stack.

7.6 Compatibility

- Support web applications.
- Support native desktop applications where the visual interaction layer can access the screen.
- Support common GUI technologies such as Electron, Qt/PySide, Java GUI and Windows applications as validation targets.
- Support different screen sizes and scaling settings within defined compatibility limits.

7.7 Security and Privacy

- Credentials, secrets and sensitive test data shall be protected.
- Passwords shall not be exposed unnecessarily in logs or reports.
- The system shall clearly communicate when screenshots or screen content are sent to cloud AI services.

- The architecture should support local or hybrid AI modes where feasible.

7.8 Explainability

For important decisions, the system should expose the selected element, confidence, relevant evidence and reason for selection. Failures should explain what was expected, what was observed, and why the system stopped.

7.9 Maintainability

Perception, reasoning, planning, execution, assertions, reporting and model providers should be modular so individual components can be replaced without redesigning the entire system.

7.10 Usability

A normal QA user should not need to understand OCR, VLMs, bounding boxes or computer vision to create and execute tests.

7.11 Observability

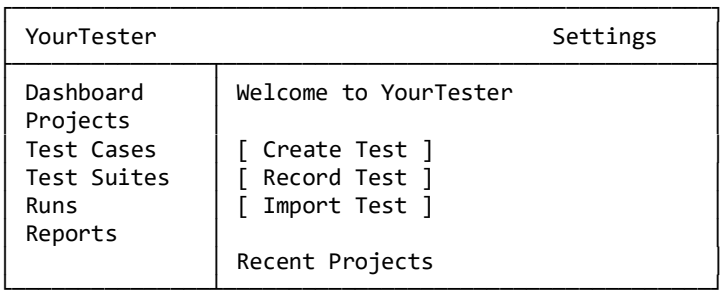
- Maintain structured execution logs.
- Record step start/end times.
- Record model decisions and confidence where appropriate.
- Record retries and healing.
- Record screenshots around important events.
- Make debugging information available without overwhelming normal users.

8. Core UX Journey

The recommended customer journey is designed around a simple loop: Describe → Plan → Review → Run → Observe → Verify → Report.

8.1 Application Launch

When the customer opens the desktop application, the first screen should be a clean dashboard rather than an AI/model configuration screen.



8.2 Create Test

The customer chooses between describing the test in natural language, recording interactions, or eventually importing existing tests.

```
Create Test

[ Describe with AI ]
Describe what you want the application to do.
```

```
[ Record interactions ]
Interact with the app and create a test from the session.
```

8.3 Natural-Language Test Description

Example: “Open the CRM, create a new customer named John Smith with email john@example.com, save it, and verify that the customer appears in the customer list.”

8.4 AI Test Plan Review

```
Test: Create Customer

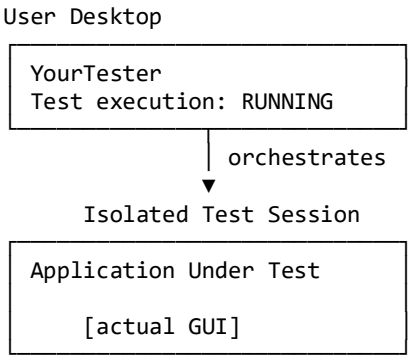
1. Launch CRM
2. Navigate to Customers
3. Click Add Customer
4. Enter "John Smith"
5. Enter "john@example.com"
6. Click Save
7. Verify success message
8. Search for "John Smith"
9. Verify customer exists

[ Edit ]     [ Run Test ]
```

The user should review the plan before execution, especially for important or destructive actions.

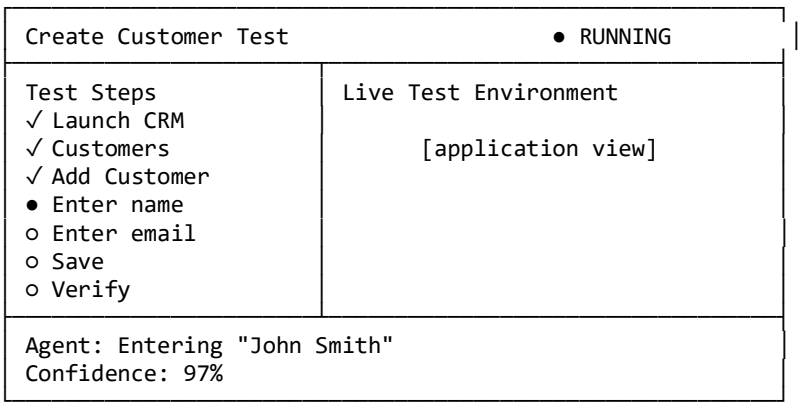
8.5 Live Execution

The recommended production architecture is to execute the test in an isolated desktop/session rather than taking over the customer's normal desktop. The main application becomes the control plane and displays a live view of the test environment.



For the earliest MVP, direct control of the user's current desktop can be supported as a development/debug mode. Production should move toward an isolated session or VM.

8.6 Execution View



8.7 AI Vision Debug View

An optional overlay should show detected element IDs, bounding boxes, labels and target selection. This is primarily a debugging and trust feature.

8.8 PASS Result

```
CREATE CUSTOMER

✓ PASSED

9 / 9 steps passed
Execution time: 14.2 sec

Step 1   ✓ Launch CRM
Step 2   ✓ Navigate Customers
Step 3   ✓ Add Customer
Step 4   ✓ Enter name
Step 5   ✓ Enter email
Step 6   ✓ Save
Step 7   ✓ Success notification
Step 8   ✓ Search customer
Step 9   ✓ Customer exists

[ View Evidence ] [ Replay ] [ Run Again ]
```

8.9 FAIL Result

```
✕ FAILED — Step 6

Action:
Click "Save"

Expected:
Customer should be created.

Observed:
"Email address is required"

AI analysis:
The email field appears empty.

Possible cause:
The email entry step may not have completed.

[ View Screenshot ]
[ View Execution ]
[ Retry Step ]
[ Edit Test ]
```

8.10 HEALED Result

```
⚠ HEALED

Original target:
"Save"

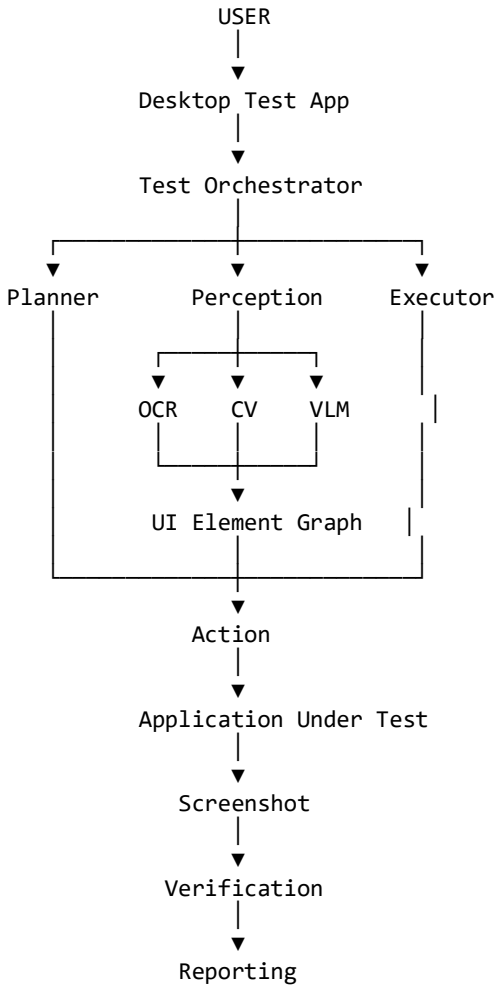
Current UI:
"Create Customer"

Recovery confidence:
92%

The test continued after recovering from a UI change.

[ Review Healing ]
```

9. Core System Architecture



10. Perception Layer

The perception layer is the “eyes” of the system. It receives screenshots and transforms pixels into a structured representation of visible UI elements, their locations, properties and relationships. It must be DOM-independent so that the same conceptual system can operate on desktop applications.

The perception layer should not be responsible for deciding what the user wants. It gathers evidence; the reasoning/planning layer determines which element satisfies the test instruction.

10.1 Perception Pipeline

```
Screenshot
   ↓
OCR
   ↓
Visual/UI region detection
   ↓
CV feature extraction
   ↓
VLM semantic interpretation
   ↓
Element fusion
```

```
   ↓
UI Element Graph
```

10.2 Element Representation

```
Element
├── stable_id
├── text
├── bbox
├── visual_role
├── interaction capability
├── confidence
├── icon/features
├── nearby text
├── parent/container
├── spatial relationships
└── current state
```

10.3 Example Element Map

```
E01: text="Customer", type=label,
     bbox=[100,100,180,125], clickable=false

E02: text="Select customer", type=dropdown,
     bbox=[190,95,400,135], clickable=true

E03: text="Customer", type=text,
     bbox=[100,180,180,205], clickable=false

E04: text="John Smith", type=text,
     bbox=[100,210,200,235], clickable=false

E05: text="", type=search_icon,
     bbox=[500,300,540,340], clickable=true
```

10.4 UI Element Taxonomy

Category	Examples
Interactive	button, link, input, textarea, checkbox, radio, dropdown, slider, tab, toggle, menu item
Non-interactive	heading, label, paragraph, table, table cell, image, icon
Containers	modal, card, form, navigation, panel

The taxonomy should not be treated as a rigid truth source. The system should collect attributes first and infer semantic role from multiple signals.

11. Resolving Duplicate Text and Ambiguity

OCR string matching alone is insufficient. The same visible text can represent a label, dropdown, table header, button, ordinary text or a completely different control.

```
Customer ▼
Customer
Customer
John Smith
```

The agent should use a combination of text, visual role, interaction capability, nearby elements, spatial relationships, instruction semantics and confidence.

```
Instruction:
"Click the Customer dropdown"

Candidate:
E01 Customer → label       0.31
E02 Customer → dropdown    0.96  ← selected
E03 Customer → text        0.18
```

The VLM should ideally choose among structured candidates rather than directly output raw screen coordinates. Once the target is selected, deterministic code calculates a safe click point from its bounding box.

12. Why the VLM Should Not Own Coordinates

A VLM can be useful for semantic interpretation but may be imprecise with small objects and pixel-level coordinates. The safer division of responsibility is:

```
VLM:
"Which candidate is the intended target?"

Deterministic executor:
"What are the target's exact pixels?"

bbox = [x1, y1, x2, y2]
safe_x = (x1 + x2) / 2
safe_y = (y1 + y2) / 2
click(safe_x, safe_y)
```

The exact click strategy should later account for safe margins, control shape, screen scaling and target-specific interaction regions.

13. DOM Independence

The product should not require a DOM because desktop applications may have no DOM. The primary perception path is screenshot-based.

```
SCREENSHOT
   ↓
OCR + CV + VLM
   ↓
UI Element Graph
   ↓
Reasoning
   ↓
Coordinate
   ↓
Mouse/Keyboard
```

Optional OS accessibility APIs, such as Windows UI Automation, may later improve role/state information for compatible applications, but they should be treated as an enhancement rather than the core dependency.

14. Action Execution Layer

The executor translates semantic actions into physical interaction.

```
CLICK(E17)
TYPE(E21, "Arjun")
SCROLL(DOWN, 600)
KEYPRESS("CTRL+A")
TYPE("hello@example.com")
DRAG(E12, E19)
```

The planner should not directly generate arbitrary low-level mouse commands. The executor should own coordinate conversion, safe click regions, timing, retries and action verification.

15. Smart Waiting

The system should avoid fixed sleeps wherever possible. It should wait for observable state changes.

- Wait until an element appears.
- Wait until an element disappears.
- Wait until a loading spinner disappears.
- Wait until a button becomes enabled.
- Wait until expected text appears.
- Wait until a dialog opens.
- Wait until the screen stabilizes.

16. Assertion and Verification System

Assertion Type	Example
Visibility	Verify Save button is visible
Presence	Verify customer appears in table
Absence	Verify error dialog disappeared
Text	Verify notification says success
Value	Verify email field contains expected value
State	Verify checkbox is checked
Visual	Verify dashboard changed to expected state
Workflow	Verify customer can be found after creation

17. Failure Taxonomy

- ELEMENT_NOT_FOUND
- AMBIGUOUS_TARGET
- ACTION_FAILED
- ASSERTION_FAILED
- APPLICATION_CRASH
- TIMEOUT
- VISION_FAILURE
- MODEL_FAILURE
- UNEXPECTED_DIALOG
- FOCUS_LOST
- UNKNOWN

18. Self-Healing

Self-healing should be controlled and observable. If the expected element is missing, the system should search for a semantically similar candidate and evaluate context.

```
Original:
"Submit"

Current:
"Create Account"

Context:
Account creation form

Result:
Likely replacement = "Create Account"
Confidence = 92%
```

The report should explicitly mark the action as HEALED so the customer knows that the UI changed and the test did not execute exactly as originally specified.

19. Execution Environment Strategy

19.1 Development Mode

Direct control of the user's current desktop is the simplest implementation for the initial prototype. It proves the core perception and execution loop quickly but is disruptive.

19.2 Production Direction

Use an isolated desktop/session so the user's normal desktop is unaffected. The main testing application becomes the control plane while a worker controls the isolated graphical environment.

```
YourTester
    │
    ▼
Test Worker
    │
    ▼
Isolated Desktop Session
    │
    ├── Application Under Test
    ├── Mouse/Keyboard
    ├── Screenshot capture
    ├── OCR/CV/VLM
    └── Test state
```

19.3 VM / Remote Worker

A virtual machine or remote worker becomes appropriate for reproducibility, isolation, parallelism and eventual cloud execution.

```
YourTester
   ├── Local Worker → isolated desktop
   ├── Remote Worker → VM
   └── Cloud Worker → VM pool
```

20. Recommended UX Principles

- Hide AI implementation details from ordinary QA users.
- Make natural-language test creation the easiest path.
- Always show the generated plan before important execution.
- Make live execution observable.
- Make failures actionable rather than merely technical.
- Expose AI vision and reasoning as optional debugging detail.
- Clearly distinguish PASS, FAIL and HEALED.
- Never silently recover from a changed UI.
- Allow manual intervention for uncertain or destructive steps.
- Keep advanced configuration available without cluttering the main workflow.

21. Suggested Desktop Application Screens

1. Dashboard / Projects
2. Project Overview
3. Create Test
4. Test Plan Editor
5. Application / Environment Setup
6. Live Execution
7. AI Vision Debugger
8. Test Result
9. Failure Investigation
10. Test Case Library
11. Test Suite Manager
12. Execution History
13. Reports / Analytics
14. Settings / Models / Execution Environment

22. Example End-to-End Scenario

A user wants to test a desktop CRM.

```
User:
"Create a customer named John Smith and verify it appears in the customer list."

AI Planner:
1. Open CRM
2. Navigate to Customers
3. Click Add Customer
4. Enter name
5. Save
6. Verify success
7. Search for John Smith
8. Verify customer exists
```

During step 3, perception may detect several occurrences of “Customer”. The system constructs candidates and identifies the interactive control based on visual and semantic context. The executor obtains its bounding box and clicks a safe interior point.

During verification, the system searches for the created customer and compares the observed UI state with the expected state. Evidence is captured for each critical step.

If the Save button changes to Create Customer, the system may propose or execute a self-healing action according to policy, then mark the test as HEALED rather than silently reporting a normal PASS.

23. Data Model Concept

```
Project
  ├── Application
  ├── Environment
  ├── TestCases
  │     ├── Steps
  │     ├── Assertions
  │     └── TestData
  ├── TestSuites
  └── Runs
        ├── StepResults
        ├── Screenshots
        ├── Logs
        ├── ModelDecisions
        └── HealingEvents
```

24. AI Responsibilities

Component	Responsibility
Planner / LLM	Convert user intent into test steps, constraints and assertions
OCR	Detect text and text bounding boxes
CV	Detect regions, shapes, visual features and candidate controls
VLM	Understand visual semantics, classify ambiguous controls and choose among candidates
Perception Fusion	Create a unified UI element graph
Executor	Perform deterministic mouse and keyboard operations
Assertion Engine	Determine whether expected state was achieved
Recovery Engine	Handle retries and possible self-healing
Reporting AI	Explain failures and summarize evidence

25. Important Engineering Principle

Do not make the VLM the entire agent. The VLM should be one component in a larger deterministic system.

```
BAD:
Screenshot → VLM → "click at (842, 73)"

BETTER:
Screenshot
  → OCR/CV/VLM
  → structured UI element graph
  → candidate selection
  → selected element
  → deterministic bounding-box calculation
  → executor
  → verification
```

26. MVP Definition

The MVP should focus on proving the complete end-to-end loop rather than implementing every enterprise feature.

- Desktop application with dashboard.
- Natural-language test creation.
- AI-generated test plan.
- Editable test steps.
- Screenshot capture.
- OCR + visual perception.
- VLM-assisted semantic element selection.
- UI element graph.
- Mouse and keyboard execution.
- Smart waiting.
- Basic assertions.
- PASS/FAIL result.
- Screenshot evidence.
- Detailed failure explanation.
- Basic test storage and rerun.

The MVP should initially support direct local execution as a development mode, then move toward isolated execution once the core agent is reliable.

27. Version Roadmap

V1 — Core AI Tester

```
Describe
→ Plan
→ Review
→ Perceive
→ Select
→ Act
→ Verify
→ Report
```

- Natural language tests
- OCR + CV + VLM perception
- UI element graph
- Mouse/keyboard execution
- Assertions
- Screenshots
- Failure reports

V2 — Reliable Tester

- Smart waiting
- Test recording
- Advanced assertions
- Failure classification
- Self-healing

- Video recording
- Test suites
- Test data
- Isolated execution

V3 — Professional QA Platform

- VM and remote workers
- Parallel execution
- CI/CD
- Scheduling
- Cross-environment testing
- Dashboards
- Team collaboration
- Permissions
- Secrets management
- Jira/GitHub/Slack integrations
- Cloud execution

28. Metrics to Benchmark

Before fixing hard product guarantees, build a benchmark dataset containing representative web and desktop screenshots, duplicate-text cases, icon-only controls, forms, dialogs, tables, different resolutions and UI changes.

Metric	What to Measure
Target identification accuracy	Correct semantic target among candidates
Bounding-box accuracy	Whether the chosen click point lies inside the intended control
Action success rate	Whether the intended interaction actually occurred
Assertion accuracy	Correct PASS/FAIL classification
Recovery success rate	Successful self-healing without incorrect actions
Flakiness rate	Inconsistent result across repeated runs
Per-step latency	Time from screenshot to verified action
End-to-end test time	Total time for a test
False recovery rate	Incorrect healing decisions
Model failure rate	Cases where AI cannot confidently decide

29. Major Technical Risks

- Small icon detection and accurate coordinate localization.
- Multiple identical text references.
- DPI scaling and coordinate transforms.
- Different screen resolutions and multi-monitor setups.
- Unexpected focus changes.
- Animations and transient UI.
- Application-specific rendering.
- VLM hallucination or imprecise localization.
- Slow cloud model inference.
- Incorrect self-healing.

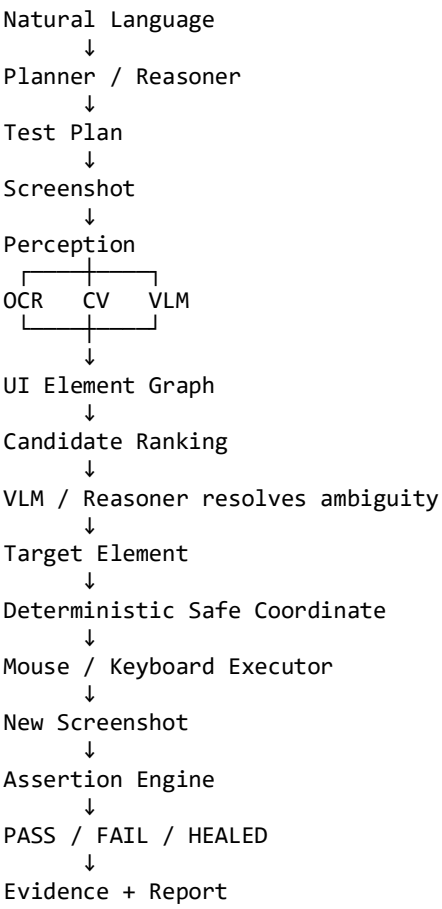
- Destructive actions performed on the wrong element.
- Credential and screenshot privacy.
- Keeping execution deterministic enough for QA.

30. Safety and Control Mechanisms

- Require explicit user confirmation for destructive actions in early versions.
- Use confidence thresholds for ambiguous visual decisions.
- Allow configurable retry limits.
- Allow a manual-pause / takeover mode.
- Show the intended target before risky actions when appropriate.
- Never hide a model uncertainty or healing event.
- Keep screenshots and logs tied to individual steps for auditability.

31. Recommended Initial Architecture Decision

The strongest architecture for the proposed product is a hybrid, DOM-independent visual agent with deterministic execution:

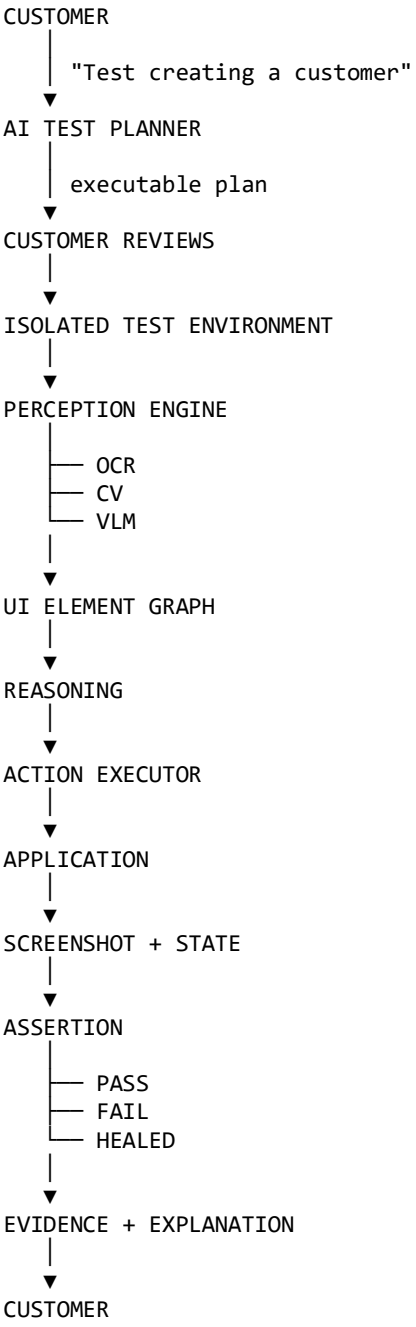


32. Product Positioning

The core differentiator is not simply AI-generated test cases. The stronger differentiator is visual, application-agnostic interaction: the agent can reason about what is visible on the screen and interact with software through human-like controls, while maintaining a structured understanding of UI elements.

This makes the architecture applicable beyond browser automation and creates a path toward desktop software, legacy software and custom GUI testing.

33. Final Product Experience



34. Immediate Build Order

15. Freeze the MVP functional requirements.
16. Define the UI Element schema and UI Element Graph.
17. Build screenshot capture and coordinate normalization.
18. Build OCR detection and bounding-box extraction.
19. Add visual candidate/region detection.
20. Introduce VLM classification for ambiguous elements and icons.
21. Build element fusion and candidate ranking.
22. Build semantic action representation.
23. Build deterministic mouse/keyboard executor.
24. Build state-based waiting.
25. Build basic assertion engine.
26. Build execution timeline and screenshot evidence.
27. Build natural-language planner.
28. Connect planner → perception → executor → assertion into a complete loop.
29. Create a benchmark suite for duplicate text, icon controls, dropdowns, forms, dialogs, resolution/DPI and UI changes.
30. Only after the core loop is reliable, implement isolated execution and self-healing.

35. Summary

The product should be built as an AI-assisted visual testing system rather than a VLM-controlled mouse. The perception layer converts screenshots into a structured UI Element Graph using OCR, computer vision and VLM reasoning. The reasoning layer maps natural-language intent to candidates and chooses the correct semantic element. The executor then calculates a deterministic safe coordinate and performs the physical interaction. The assertion layer verifies the result, while recovery and reporting turn failures into actionable information.

For the first version, focus on the complete loop: Describe → Plan → Perceive → Identify → Act → Verify → Explain. Once this loop is reliable, expand into isolated execution, self-healing, regression suites, CI/CD, parallel workers and enterprise functionality.